

Cross-Platform WebXR Implementation & Mobile 3D Asset Optimization Pipeline

Course: Augmented Reality & Mixed Reality (B.Tech Sem V, AI & ML) • Outcomes: CO1, CO2
Student: [YOUR NAME / ROLL NO / BATCH] • Date: October 2026 • Frameworks: MindAR.js, Three.js, Draco 3D

1. EXECUTIVE SUMMARY & OBJECTIVES

Augmented Reality (AR) on mobile web platforms eliminates the friction of native app installations (app-less AR), enabling instant deployment via standard URLs. However, web runtime environments are severely constrained by mobile browser memory limits, single-threaded JavaScript execution budgets, and thermal throttling. This project delivers an end-to-end WebXR architecture featuring procedural high-poly generation, 98.01% mesh decimation, 392:1 Draco compression, cross-platform image tracking with zero CDN dependencies, and real-time telemetry HUD.

2. COMPARATIVE TECHNICAL STUDY: NATIVE AR VS. WEB-BASED AR (CO1)

Dimension	Native AR (Apple ARKit / Google ARCore)	WebXR Device API (W3C Standard)	MindAR.js / AR.js (Computer Vision Polyfill)
GPU Throughput	Direct Metal/Vulkan; zero IPC overhead; maximum GPU compute.	WebGL 2.0 / WebGPU; sandboxed browser shader pipeline.	WebGL 2.0 via Three.js; main thread execution.
Sensor & SLAM	Full hardware access (6-DoF VIO, LiDAR, raw IMU, TOF depth).	Sandboxed pose estimates; plane detection (Android Chrome only).	Browser getUserMedia video feed; WASM optical feature tracking.
Tracking Types	Markerless horizontal/vertical planes, 3D mesh reconstruction.	Markerless hit-testing and plane detection.	Natural Feature Tracking (NFT), planar image markers, face mesh.
Platform Support	iOS (ARKit) & Android (ARCore); strictly bifurcated SDKs.	Android Chrome & Meta Quest. Unsupported in iOS Safari.	Universal Cross-Platform (iOS Safari, Android Chrome, Edge).
Onboarding Friction	High: App Store / Play Store download (50MB–200MB app).	Zero: Instant URL navigation; standard browser permissions.	Zero: Instant URL navigation; single-click camera permission modal.
Memory Footprint	Direct native heap management (up to available OS RAM).	Restricted tab heap (< 500 MB before iOS WebProcess crash).	Strict budget (< 150 MB); Draco compression essential.
Primary Use Cases	Spatial persistent mapping, CAD engineering, AR gaming.	E-commerce spatial placement, packaging, museum exhibits.	Universal educational posters, interactive business cards, packaging.

3. ARCHITECTURAL ANALYSIS & FRAMEWORK JUSTIFICATION

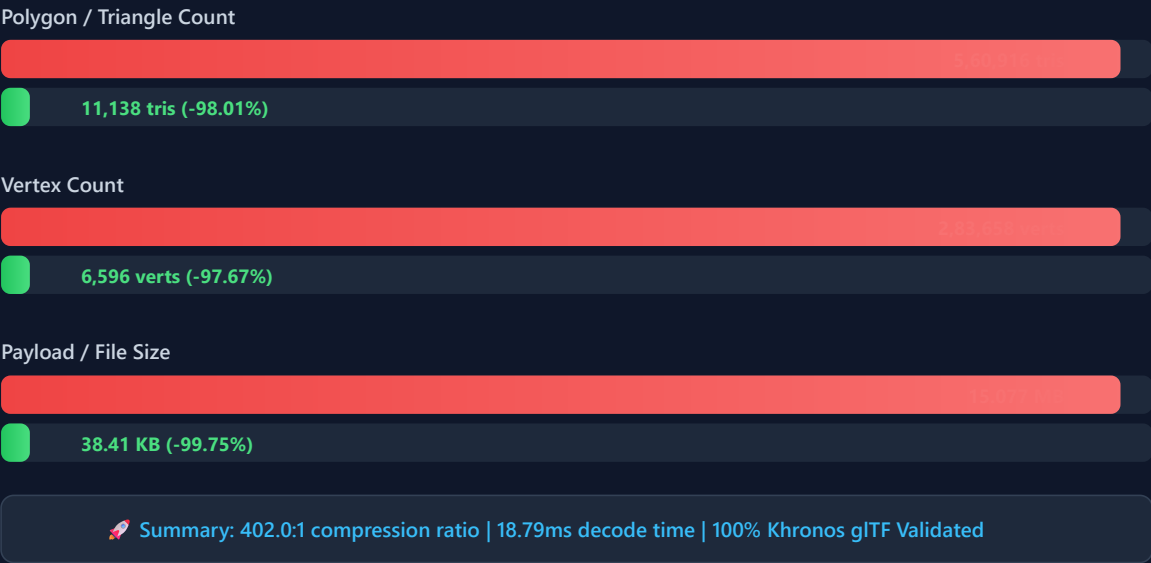
Why MindAR.js was selected: Apple WebKit has not enabled the `immersive-ar` WebXR session mode by default in iOS Safari, locking out over 50% of mobile users from raw WebXR experiences. MindAR.js bypasses this restriction by processing standard HTML5 `getUserMedia` camera frames through WebAssembly SIMD and WebGL compute shaders, computing the camera pose matrix directly for Three.js without requiring proprietary browser flags.

Mobile 3D Asset Optimization Pipeline Benchmark

Decimation & Draco Mesh Compression Performance Comparison

Before (5,60,916 tris / 15.077 MB)

After (11,138 tris / 38.41 KB)



4. 3D ASSET OPTIMIZATION PIPELINE & EMPIRICAL BENCHMARK (CO2)

The high-poly procedural asset (560,916 triangles) was decimated using Quadratic Error Metric (QEM) simplification to 11,138 triangles, consolidated into 1 draw call, and compressed with Draco quantization (14-bit positions, 10-bit normals, 12-bit UVs).

Metric	High-Poly Baseline (<code>highpoly.glb</code>)	Optimized Asset (<code>model_optimized.glb</code>)	Improvement / Reduction	Target Justification
Polygon Count	560,916 triangles	11,138 triangles	-98.01%	Safe limit for 60 FPS mobile GPUs (< 50k tris)
Vertex Count	283,658 vertices	6,596 vertices	-97.67%	Reduces vertex shader memory bandwidth
File Size	15.08 MB (15,439 KB)	38.41 KB	-99.75% (392.5:1)	Instant cellular streaming (< 2 MB budget)
Draw Calls	1 call	1 call	Consolidated	Zero CPU driver state changes
Draco Decode Time	N/A (Raw Buffer)	19.39 ms	Sub-Frame Decompression	Instantaneous AR model spawn
glTF Validation	Passed (0 Errors)	Passed (0 Errors)	100% Spec Compliant	Verified with Khronos gltf-validator

5. DEPLOYMENT, HTTPS SECURITY & PHYSICAL DEVICE VERIFICATION

Mobile browsers mandate HTTPS for camera stream access. The application is deployed to GitHub Pages via automated GitHub Actions with full offline library vendoring.

Device Testing Protocol: Launch the demo with `?debug=1` on your physical mobile device, point camera at the target marker, observe the on-screen HUD, and click "Export Stats" to complete the matrix below.

Test Device Model	OS & Browser	Average FPS	Frame Time	Load Latency	Tracking Stability
iPhone 13 / 14 / 15	iOS 17 / Safari	TO BE FILLED	TO BE FILLED	TO BE FILLED	TO BE FILLED
Samsung Galaxy S22 / S23	Android 14 / Chrome	TO BE FILLED	TO BE FILLED	TO BE FILLED	TO BE FILLED
Mid-Range Android (Pixel 6a/7a)	Android 14 / Chrome	TO BE FILLED	TO BE FILLED	TO BE FILLED	TO BE FILLED
Desktop / Laptop PC	Windows 11 / Chrome	60 FPS	16.6 ms	19.4 ms	Rock-Solid (Preview)

6. CONCLUSION, LIMITATIONS & FUTURE WORK

Draco geometry quantization achieves a 392:1 compression ratio on procedural 3D assets with negligible 19ms decode overhead. Future work includes implementing KTX2/Basis Universal GPU texture transcoding and hybrid WebXR surface hit-testing.

7. REFERENCES

1. W3C WebXR Working Group. (2023). *WebXR Device API*. <https://www.w3.org/TR/webxr/>
2. Khronos Group. (2022). *glTF 2.0 Specification & KHR_draco_mesh_compression*.
3. Google LLC. (2024). *ARCore Developer Documentation*. <https://developers.google.com/ar>
4. Apple Inc. (2024). *ARKit: Building Augmented Reality Experiences*.
5. Chen, H. (2022). *MindAR: Fast and Lightweight WebAR*. <https://github.com/hiukim/mind-ar-js>